Informationstechnologie I – Skript zu Termin 5: Strings und Textverarbeitung

Dieses Skript erläutert die Begriffe und Werkzeuge aus den Folien. Die Übungen stehen gesammelt in Aufgaben_5.md. Die drei Übungsblöcke folgen auf die jeweils benötigten Grundlagen. Die Beispiele erklären einzelne Werkzeuge; in den Aufgaben kombinieren Sie diese selbst zu einem Programm.

Lernziele

Nach diesem Termin können Sie den Zusammenhang zwischen Zeichen, Codepoints und Codierungen erklären, `char` und `string` unterscheiden und Texte mit Indizes und Schleifen untersuchen. Sie beachten die Unveränderlichkeit von Strings, bereinigen Eingaben und nutzen Such-, Teiltext- und Ersetzungsoperationen.

Vorausgesetzt werden Variablen, Bedingungen, Schleifen und Methoden. Führen Sie Beispiele einzeln im vorbereiteten Projekt aus. Ersetzen Sie den bisherigen Übungscode, damit Variablennamen nicht mehrfach deklariert werden.

1. Zeichen, Zahlen und Codierungen

Texte begegnen uns etwa in Benutzereingaben, Artikelbezeichnungen und Protokolldateien. Damit ein Computer sie speichern und verarbeiten kann, braucht es gemeinsame Regeln für ihre Darstellung als Zahlen und Bytes.

Drei Ebenen sind dabei zu unterscheiden:

- Ein **Zeichensatz** legt Zeichen und zugehörige Nummern fest. Bei Unicode heißt eine solche Nummer **Codepoint**.
- Eine **Codierung** legt fest, wie Codepoints als Codeeinheiten bzw. Bytes dargestellt werden. Beispiele sind UTF-8, UTF-16 und UTF-32.
- Ein **Graphem** entspricht einem vom Benutzer als zusammengehörig wahrgenommenen Zeichen. Es kann aus mehreren Codepoints bestehen.

So kann ein sichtbares `ü` als einzelner Codepoint oder als `u` gefolgt von einem kombinierenden Umlautzeichen vorliegen. Sichtbares Zeichen, Codepoint und gespeicherte Einheit sind deshalb nicht immer dasselbe.

ASCII: 128 Zeichen

ASCII (American Standard Code for Information Interchange) ist ein 7-Bit-Code mit $2^7 = 128$ möglichen Werten von 0 bis 127. Dazu gehören Steuerzeichen, Ziffern, grundlegende lateinische Buchstaben und Satzzeichen.

Dezimal	Zeichen / Bereich	Bedeutung
0–31	z. B. 9: Tab, 10: LF, 13: CR	Steuerzeichen
32	Leerzeichen	Abstand

Dezimal	Zeichen / Bereich	Bedeutung
33–47	z. B. `!`, `"`, `#`	Satz- und Sonderzeichen
48–57	`0` … `9`	Ziffern
58–64	z. B. `:`, `?`, `@`	Satz- und Sonderzeichen
65–90	`A` … `Z`	Großbuchstaben
91–96	z. B. `[`, `]`, `_`	Sonderzeichen
97–122	`a` … `z`	Kleinbuchstaben
123–126	z. B. `{`, `}`	Sonderzeichen
127	DEL	Steuerzeichen

ASCII enthält keine deutschen Umlaute, kein Eurozeichen und keine Emojis. Die Bezeichnung „erweitertes ASCII“ steht nicht für einen einzigen verbindlichen 256-Zeichen-Standard.

```
Zeichen      Codewert      Als Byte dargestellt
   A    →       65    →         01000001
   B    →       66    →         01000010
```

Die zusätzliche führende Null zeigt hier die Darstellung in einem 8-Bit-Byte; der ASCII-Wertebereich benötigt nur 7 Bit.

Zeichenwert und Zahlenwert

In C# können Sie den numerischen Wert einer `char`-Codeeinheit durch einen Cast sichtbar machen:

```
char buchstabe = 'A';
Console.WriteLine((int)buchstabe); // 65
Console.WriteLine((char)66);       // B

char ziffer = '7';
Console.WriteLine((int)ziffer);    // 55
```

`(int)` wandelt den `char`-Wert in eine ganze Zahl um. `(char)` interpretiert hier die Zahl als UTF-16-Codeeinheit. Das ist keine Umwandlung eines eingegebenen Zahlentexts in dessen Zahlenwert: `'7'` hat den Codewert 55. Die Werte `'7'`, `"7"` und `7` haben die Typen `char`, `string` und `int`.

2. Unicode und seine Codierungen

Unicode umfasst Zeichen aus vielen Schriftsystemen sowie Symbole und Emojis. Die ersten 128 Codepoints stimmen mit ASCII überein.

Zeichen	Unicode-Codepoint	In ASCII enthalten?
A	U+0041	Ja
ä	U+00E4	Nein
€	U+20AC	Nein
中	U+4E2D	Nein
😀	U+1F600	Nein

Die Notation `U+` wird von einer hexadezimalen Zahl gefolgt. Hexadezimale Zahlen verwenden die Ziffern 0–9 und A–F. Beispielsweise entspricht `U+0041` dem Dezimalwert 65. Die Nummer identifiziert den Codepoint; wie das Zeichen auf dem Bildschirm aussieht, hängt unter anderem von der verwendeten Schrift ab.

UTF-8, UTF-16 und UTF-32

Die Codierung bestimmt die gespeicherte Darstellung eines Unicode-Skalarwerts. Ein Skalarwert ist ein Unicode-Codepoint außerhalb des für Surrogate reservierten Bereichs.

Codierung	Darstellung pro Skalarwert	`A`	😀
UTF-8	1–4 Bytes	1 Byte	4 Bytes
UTF-16	1–2 Codeeinheiten mit je 16 Bit	2 Bytes	4 Bytes
UTF-32	1 Codeeinheit mit 32 Bit	4 Bytes	4 Bytes

Diese Größen beziehen sich auf den codierten Inhalt, ohne Verwaltungsdaten oder Dateikopf. Ein Byte hat 8 Bit.

C#-Strings sind Folgen von **UTF-16-Codeeinheiten**. Eine eingelesene oder geschriebene Datei kann dennoch UTF-8 verwenden: Beim Lesen werden Bytes decodiert, beim Schreiben wird der Text in Bytes encodiert. Werden Bytes mit einer unpassenden Codierung gelesen, kann eine fehlerhafte Darstellung entstehen.

3. Die Datentypen `char` und `string`

Ein `char` speichert genau **eine UTF-16-Codeeinheit** mit 16 Bit. Ein `string` ist eine Folge solcher Codeeinheiten.

```
char buchstabe = 'A';
char euro = '€';
string text = "Hallo Welt";
string einZeichen = "A";
string leer = "";
```

`char`-Literale stehen in einfachen, String-Literale in doppelten Anführungszeichen. `'A'` und `"A"` sehen in der Ausgabe gleich aus, haben aber verschiedene Typen. `""` ist ein gültiger leerer String; ein leerer `char`

oder ein `char`-Literal mit mehreren Codeeinheiten ist nicht erlaubt.

Ein sichtbares Zeichen kann mehrere Codeeinheiten benötigen

Viele gebräuchliche Zeichen wie `A` oder `€` passen in eine einzelne UTF-16-Codeeinheit. Codepoints außerhalb der Basic Multilingual Plane, beispielsweise 😀 , benötigen zwei Codeeinheiten. Diese Kombination heißt **Surrogate-Pair**.

```
string wort = "Hi";
string emoji = "😀";
Console.WriteLine(wort.Length);  // 2
Console.WriteLine(emoji.Length); // 2
```

```
Text      UTF-16-Codeeinheiten         Length
"Hi"      [ H ][ i ]                      2
"😀"      [ D83D ][ DE00 ]                2
             zusammen ein Emoji
```

`Length` zählt Codeeinheiten, nicht sichtbare Zeichen. Auch `"e\u0301"`, also `e` mit einem kombinierenden Akzent, hat die Länge 2. Für die Verarbeitung von Graphemen bietet .NET beispielsweise `System.Globalization.StringInfo`. In den Umkehr- und Palindromaufgaben verwenden wir einfache Buchstaben; das Umordnen einzelner `char`-Werte kann komplexere Zeichen beschädigen.

Escape-Sequenzen

Ein Backslash leitet in normalen C#-Literalen eine Escape-Sequenz ein:

Schreibweise	Bedeutung
`\n`	Zeilenumbruch (LF)
`\t`	Tabulator
`\"`	Doppeltes Anführungszeichen im String
`\\`	Backslash
`\u20AC`	UTF-16-Codeeinheit für `€`, vier Hex-Ziffern
`\U0001F600`	Codepoint für 😀 , acht Hex-Ziffern

```
Console.WriteLine("Er sagt: \"Hallo!\"\nNächste Zeile.");
char euro = '\u20AC';
string emoji = "\U0001F600";
```

Die allgemeine Codepoint-Notation `U+20AC` und die Schreibweise `\u20AC` im C#-Literal beschreiben dasselbe Eurozeichen auf unterschiedliche Weise. Ein `\U`-Escape verlangt acht Hex-Ziffern und kann

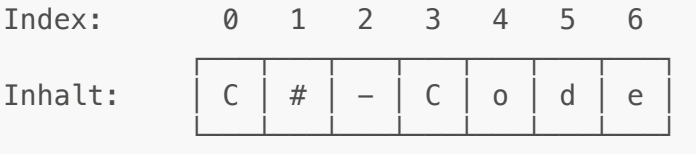
auch kleinere Codepoints darstellen.

4. Länge, Indexzugriff und Eingaben

Zählen ab 0

Die Eigenschaft `Length` liefert die Anzahl der Codeeinheiten. Der Zugriff mit eckigen Klammern liefert einen einzelnen `char`-Wert. Indizes beginnen bei 0.

```
string text = "C#-Code";
char erstes = text[0];                 // 'C'
char letztes = text[text.Length - 1];  // 'e'
```



Für einen nicht leeren String gilt `0 <= index && index < text.Length`. Der letzte Index ist `Length - 1`. `text[text.Length]` ist immer außerhalb des gültigen Bereichs. Ein ungültiger Zugriff führt zur Laufzeit zu einer `IndexOutOfRangeException`.

```
string text = "";
if (text.Length > 0)
{
    Console.WriteLine(text[text.Length - 1]);
}
else
{
    Console.WriteLine("Der Text ist leer.");
}
```

Bei einem leeren String gibt es keinen gültigen Index, auch nicht 0. Die Prüfung muss vor dem Zugriff erfolgen. Beachten Sie außerdem: `Length` ist eine Eigenschaft ohne runde Klammern; Methoden werden mit Klammern aufgerufen.

Leerer String, Leerzeichen und `null`

```
Console.Write("Text: ");
string text = Console.ReadLine() ?? "";
Console.WriteLine($"Länge: {text.Length}");
```

`Console.ReadLine()` liefert eine eingegebene Zeile ohne abschließenden Zeilenumbruch. Enter ohne Inhalt ergibt `""`; bei Eingabeende kann das Ergebnis `null` sein. Der Operator `??` verwendet den rechten Wert, wenn der linke `null` ist. Mit `?? ""` steht danach immer ein String für die weitere Verarbeitung zur Verfügung.

Wert	Bedeutung	Folge
`""`	Leerer String	Länge 0
`" "`	Ein Leerzeichen	Länge 1
`null`	Kein String vorhanden	Zugriff auf `.Length` führt zu einem Fehler

Das Eingabemuster behandeln wir in den Aufgaben als Standard. Ob eine leere Eingabe inhaltlich erlaubt ist, entscheidet die jeweilige Aufgabenstellung.

5. Unveränderlichkeit von Strings

Ein String-Objekt ist **unveränderlich (immutable)**. Eine Textoperation schreibt seinen Inhalt nicht um, sondern liefert einen Ergebniswert zurück. Wird ein anderer Text benötigt, kann die Variable anschließend auf diesen verweisen.

```
string text = "Hallo";
string kopie = text;
text.ToUpper();
Console.WriteLine(text); // Hallo

text = text.ToUpper();
Console.WriteLine(text);  // HALLO
Console.WriteLine(kopie); // Hallo
```

Der erste Aufruf berechnet ein Ergebnis, verwendet es aber nicht weiter. Beim zweiten Aufruf wird das Ergebnis zugewiesen. Dabei gilt dieselbe Regel wie bei Berechnungen mit Zahlen: erst die rechte Seite auswerten, dann die linke Variable aktualisieren.



Mehrere Variablen können denselben unveränderlichen Text verwenden. Eine spätere Zuweisung an eine Variable verändert den Text der anderen nicht. Auch `text[0] = 'h';` ist nicht erlaubt: Der String-Indexer ist nur lesbar.

Merksatz: Wenn Sie mit dem Ergebnis einer String-Methode weiterarbeiten möchten, müssen Sie es verwenden: etwa zuweisen, ausgeben, vergleichen oder an eine weitere Methode übergeben. Nicht jeder Methodenaufruf muss ein neues Objekt anlegen; ein unverändertes Ergebnis kann die vorhandene Instanz wiederverwenden.

6. Strings durchlaufen und Zeichen untersuchen

`foreach`: die Werte lesen

```
string text = "Code";
foreach (char zeichen in text)
{
    Console.WriteLine(zeichen);
}
```

Die Ausgabe enthält `C`, `o`, `d`, `e` auf jeweils einer Zeile. Pro Durchlauf erhält die Laufvariable die nächste Codeeinheit. Ein eigener Index ist nicht nötig. Ein leerer String führt zu null Durchläufen.

`for`: die Position mitverwenden

```
string text = "Code";
for (int i = 0; i < text.Length; i++)
{
    Console.WriteLine($"Index {i}: {text[i]}");
}
```

Der Index nimmt die Werte 0, 1, 2 und 3 an. Bei `i == 4` ist die Bedingung falsch und es erfolgt kein Zugriff mehr. `i <= text.Length` wäre hier falsch, weil es den ungültigen Index 4 einschließt.

Eine `for`-Schleife eignet sich, wenn Position, Startwert oder Schrittweite eine Rolle spielen. Beide Schleifenarten arbeiten hier mit Codeeinheiten; ein Emoji kann deshalb auf zwei Durchläufe verteilt werden.

Zeichen mit `char`-Methoden prüfen

Aufruf	Ergebnis	Zweck
`char.IsDigit('7')`	`true`	Dezimalziffer erkennen
`char.IsLetter('ä')`	`true`	Buchstaben erkennen
`char.IsUpper('A')`	`true`	Großbuchstaben erkennen
`char.IsWhiteSpace(' ')`	`true`	Whitespace erkennen
`char.ToLower('E')`	`'e'`	Kleinbuchstaben zurückgeben

Die `Is...`-Methoden liefern `bool`, `char.ToLower` liefert `char`. Die Methoden werden über den Typnamen `char` aufgerufen. Dagegen arbeitet `text.ToLower()` auf einem String und liefert einen String.

`char.IsDigit` erkennt auch Unicode-Dezimalziffern außerhalb von `0` bis `9`. Verlangt eine Aufgabe ausschließlich ASCII-Ziffern, prüfen Sie den Bereich mit `zeichen >= '0' && zeichen <= '9'`.

Übungsblock 1: Zeichen untersuchen

Bearbeiten Sie jetzt die Aufgaben **1–3** im Aufgabenblatt: Postleitzahl prüfen, Vokale zählen und Passwortanalyse. Dabei verbinden Sie Längenprüfung, Zeichenbedingungen und Schleifen. Planen Sie vor dem Programmieren, welche Werte während der Schleife erhalten bleiben müssen, und testen Sie die Kriterien unabhängig voneinander.

7. Texte aufbauen und bereinigen

Konkatenation mit `+`

```
string vorname = "Max";
string nachname = "Mustermann";
string vollerName = vorname + " " + nachname;
Console.WriteLine("Benutzer: " + vollerName);
```

Konkatenation bedeutet Verketten. Das Leerzeichen zwischen den Namen wird ausdrücklich als Text hinzugefügt. Die Variablen `vorname` und `nachname` werden dadurch nicht verändert.

Der Operator `+` kann sowohl addieren als auch Text verketten. Entscheidend sind die beteiligten Typen und die Auswertungsreihenfolge:

```
Console.WriteLine(2 + 3 + " Punkte");   // 5 Punkte
Console.WriteLine("Punkte: " + 2 + 3);  // Punkte: 23
Console.WriteLine("Punkte: " + (2 + 3)); // Punkte: 5
```

Diese Ausdrücke werden unter Beachtung der Klammern von links nach rechts ausgewertet. Bei `"Punkte: " + 2` entsteht bereits ein String; die folgende `3` wird angehängt. Dagegen ergibt `char + char` eine numerische Addition, keine Textverkettung.

Bei wiederholten Verkettungen in Schleifen können viele Zwischenwerte und Kopieroperationen entstehen. Für kurze Übungstexte ist der einfache Aufbau ausreichend. Für umfangreiches Zusammensetzen bietet .NET unter anderem `StringBuilder`.

Interpolation mit `$`

```
string name = "Anna";
int anzahl = 3;
Console.WriteLine($"{name} hat {anzahl} Nachrichten.");
Console.WriteLine($"Morgen sind es {anzahl + 1}.");
```

Interpolation setzt Werte oder Ergebnisse von Ausdrücken in `{...}` ein. Das vorangestellte `$` aktiviert diese Schreibweise. Ohne `$` werden die geschweiften Klammern und Namen wörtlich ausgegeben.

Groß-/Kleinschreibung

```
string original = "MischText";
string gross = original.ToUpper(); // "MISCHTEXT"
string klein = original.ToLower(); // "mischtext"
Console.WriteLine(original);       // MischText
```

Stringvergleiche mit `==` unterscheiden Groß- und Kleinschreibung: `"Code" == "code"` ergibt `false`. Die Umwandlung in eine gemeinsame Schreibweise kann einfache Eingaben für die weitere Verarbeitung vorbereiten.

`ToUpper()` und `ToLower()` verwenden standardmäßig die aktuelle Kultur. Für technische Vergleiche ohne Beachtung der Groß-/Kleinschreibung gibt es beispielsweise `string.Equals(a, b, StringComparison.OrdinalIgnoreCase)`. Die Umwandlung in Kleinbuchstaben ersetzt keine allgemeine Unicode-Normalisierung.

Whitespace an den Rändern entfernen

```
string eingabe = "   Hallo Welt   ";
string bereinigt = eingabe.Trim();
Console.WriteLine($"[{bereinigt}]"); // [Hallo Welt]
```

`Trim()` entfernt Whitespace am Anfang und Ende, einschließlich Leerzeichen, Tabulatoren und Zeilenumbrüchen. Abstände im Inneren bleiben erhalten. `" ".Trim()` ergibt den leeren String.

```
Vorher:   [···Hallo·Welt···]
Nachher:  [Hallo·Welt]
           · markiert hier ein Leerzeichen
```

Verarbeitungsschritte verketten

```
string eingabe = "  SCHRAUBE  ";
string bereinigt = eingabe.Trim();
string normalisiert = bereinigt.ToLower();
```

Die Verarbeitung lässt sich als Folge von Ergebniswerten lesen:

```
"  SCHRAUBE  " → Trim() → "SCHRAUBE" → ToLower() → "schraube"
```

Alternativ können Sie die Aufrufe verketten:

```
string normalisiert = eingabe.Trim().ToLower();
```

Die beiden Schreibweisen sind Alternativen und werden nicht mit derselben Variablendeklaration zusammen eingefügt. Die ursprüngliche Variable `eingabe` bleibt in beiden Fällen unverändert. „Normalisieren“ meint hier die Vereinheitlichung für einen konkreten Anwendungsfall, nicht die Unicode-Operation `Normalize()`.

Übungsblock 2: Texte aufbauen und vergleichen

Bearbeiten Sie jetzt die Aufgaben **4–6**: Text umkehren, Antworten vereinheitlichen und Palindrom-Prüfung. Die Werkzeuge sind bekannt; Reihenfolge, Bedingungen und Schleifenlogik entwickeln Sie selbst. Eine Skizze mit Indizes hilft bei Aufgaben, in denen Zeichen in einer anderen Reihenfolge gelesen oder miteinander verglichen werden.

8. Textstellen finden und verarbeiten

`IndexOf`: erste Fundstelle suchen

```
string text = "Hallo Welt, schöne Welt!";
int position = text.IndexOf("Welt"); // 6
int fehlt = text.IndexOf("Erde");    // -1
```

`IndexOf` liefert den Startindex des ersten Treffers. Ein Treffer kann auch an Position 0 liegen. `-1` bedeutet, dass der Suchtext nicht gefunden wurde. Prüfen Sie deshalb `position >= 0`, wenn Sie wissen möchten, ob ein Treffer vorliegt.

```
Index:   0 1 2 3 4 5 6 7 8 9 ...
Text:    H a l l o · W e l t ...
                     ↑ erster Treffer
```

Der gezeigte Aufruf mit einem String sucht standardmäßig kulturabhängig. Für technische String-Suchen lässt sich die Überladung mit `StringComparison.Ordinal` verwenden. Mit `IndexOf('#')` können Sie nach einem einzelnen `char` suchen.

`Substring`: Teiltext ausschneiden

```
string text = "Programmieren";
string anfang = text.Substring(0, 4); // "Prog"
string ende = text.Substring(8);      // "ieren"
```

`Substring(start)` liefert den Text ab dem Startindex bis zum Ende. `Substring(start, länge)` liefert eine bestimmte Anzahl von Codeeinheiten. Der zweite Parameter ist **kein Endindex**.

```
Index:    0 1 2 3 4 5 6 7 8 9 10 11 12
Text:     P r o g r a m m i e  r  e  n
          └─────┘         └──────────┘
          Start 0         Start 8 bis Ende
          Länge 4
```

Start und Länge dürfen nicht negativ sein; der Start darf höchstens `Length` betragen und `start + länge` darf nicht hinter dem Ende liegen. Ungültige Argumente führen zu einer `ArgumentOutOfRangeException`. Anders als beim einzelnen Indexzugriff ist `text.Substring(text.Length)` erlaubt und liefert `""`.

Suchergebnisse absichern

```
string notiz = "Raum #204";
int position = notiz.IndexOf('#');
if (position >= 0)
{
    Console.WriteLine($"Markierung an Position {position}.");
}
else
{
    Console.WriteLine("Keine Markierung gefunden.");
}
```

Das Muster lautet: suchen, Rückgabewert prüfen, abhängig davon weiterarbeiten. Weitere Prüfungen hängen von der anschließenden Operation ab. Ein gültiger Treffer allein sagt beispielsweise noch nicht, dass vor oder nach der Fundstelle weiterer Text vorhanden ist.

`Replace`: Vorkommen ersetzen

```
string text = "Ein Tag ist ein Tag.";
string ersetzt = text.Replace("Tag", "Moment");
Console.WriteLine(ersetzt); // Ein Moment ist ein Moment.
Console.WriteLine(text);    // Ein Tag ist ein Tag.
```

Die gezeigte Überladung ersetzt alle exakt passenden Vorkommen und unterscheidet Groß- und Kleinschreibung. Sie prüft keine Wortgrenzen: Der Suchtext `"Tag"` passt auch in `"Tagung"`. Der alte Suchtext darf nicht leer sein; eine leere Suche löst eine `ArgumentException` aus.

Der Originaltext bleibt unverändert. Möchten Sie Original und Ergebnis vergleichen oder beide ausgeben, bewahren Sie sie in getrennten Variablen auf.

Übungsblock 3: Textteile verarbeiten

Bearbeiten Sie jetzt die Aufgaben **7–9**: E-Mail-Adresse zerlegen, Suchtext ersetzen und Wörter zählen. Verbinden Sie die Werkzeuge mit eigenen Prüfungen. Legen Sie vorab fest, welche Eingaben gültig sind und was bei fehlenden Trennzeichen, leeren Suchtexten oder mehreren Leerzeichen passieren soll. Die vereinfachten Regeln aus dem Aufgabenblatt bestimmen das erwartete Verhalten.

9. Werkzeugübersicht und typische Fehler

Zweck	Werkzeug	Ergebnis
Anzahl der Codeeinheiten	`text.Length`	`int`
Einzelne Codeeinheit lesen	`text[i]`	`char`
Codeeinheiten durchlaufen	`foreach` oder `for`	Verarbeitung in der Schleife
Groß-/Kleinschreibung ändern	`ToUpper()`, `ToLower()`	`string`
Ränder bereinigen	`Trim()`	`string`
Erste Fundstelle suchen	`IndexOf(...)`	`int`, bei fehlendem Treffer `-1`
Teiltext ausschneiden	`Substring(...)`	`string`
Vorkommen ersetzen	`Replace(...)`	`string`

Fehler	Ursache	Vorgehen
`text.Length()`	Eigenschaft wie Methode verwendet	`text.Length` schreiben
`text[text.Length]`	Index hinter dem Ende	Vorher prüfen, dann ggf. `Length - 1` nutzen
`i <= text.Length` beim Vorwärtsdurchlauf	Ein Durchlauf zu viel	`i < text.Length` verwenden
`text.Trim();` ohne weitere Verwendung	Ergebnis verworfen	Ergebnis zuweisen oder direkt verwenden
`position > 0` für „gefunden“	Treffer am Anfang übersehen	`position >= 0` prüfen
`Substring(2, 5)` als Bereich 2 bis 5 lesen	Anzahl mit Endindex verwechselt	5 Codeeinheiten ab Index 2
Sichtbares Zeichen mit einem `char` gleichsetzen	UTF-16-Darstellung übersehen	Codeeinheiten, Codepoints und Grapheme unterscheiden

Unterscheiden Sie beim Prüfen eines Programms Compilerfehler, Laufzeitfehler und falsche Ergebnisse. Überlegen Sie zuerst die erwartete Ausgabe und vergleichen Sie diese anschließend mit dem tatsächlichen Verhalten. Nutzen Sie neben Erfolgsfällen auch leere Eingaben, Grenzfälle und Eingaben, die einzelne Bedingungen gezielt verletzen.